

Propuesta de enunciados para las presentaciones orales

Índice

1. Análisis de Trayectoria Balística con resistencia aerodinámica	2
2. Análisis Numérico y Telemetría del Rendimiento de un Turbopropulsor	5
3. Telemetría y Dinámica de Vuelo del Cohete Sonda “AeroPy-I”	8
4. Senda de Aterrizaje y Dinámica de Frenado de una Aeronave Comercial	11
5. Análisis Orbital y de Telecomunicaciones de la Constelación Starlink	14
6. Análisis Telemétrico y Aerodinámico del Lanzamiento de una Sonda a Marte	17
7. Análisis de Telemetría y Ascenso de una Sonda Meteorológica Estratosférica	20

1. Análisis de Trayectoria Balística con resistencia aerodinámica

Contexto Físico y Objetivo

Sois el equipo de dinámica de vuelo y telemetría de la misión **ÍCARO**, encargada de probar un nuevo diseño de sonda atmosférica. Esta sonda se lanza mediante un tiro parabólico, e incorpora un sistema de freno aerodinámico pasivo para limitar su velocidad de impacto.

Vuestro objetivo como ingenieros es desarrollar un *script* integral en Python que procese los datos en bruto obtenidos durante el primer vuelo de prueba. Deberéis calibrar los sensores de lanzamiento, caracterizar el perfil aerodinámico de la sonda a partir de ensayos en túnel de viento, calcular el estado atmosférico, procesar la telemetría del radar para obtener la cinemática del vuelo, y finalmente, estimar el trabajo disipado por la atmósfera y el instante exacto del impacto.

Datos de Partida

Para inicializar vuestro código, deberéis importar las librerías científicas de Python y cargar los siguientes arrays con los datos brutos recopilados por los equipos de instrumentación:

```
1      import numpy as np
2
3      # 1. Matriz de acoplamiento del acelerometro 3D y voltajes medidos (mV)
4      M_calib = np.array([[1.05,0.02,-0.01], [0.01,0.98,0.03], [-0.02,0.01,1.02]])
5      V_medido = np.array([15.2, -4.5, 98.1])
6
7      # 2. Ensayos en tunel de viento: Presion dinamica (Pa) y Fuerza Drag (N)
8      q_tunel = np.array([500, 1000, 1500, 2000, 2500, 3000])
9      F_drag_tunel = np.array([12.1, 25.3, 37.8, 51.0, 62.5, 76.2])
10
11     # 3. Atmosfera Estandar (ISA): Altitud (m) y Densidad (kg/m^3)
12     altitudes_isa = np.array([0, 500, 1000, 1500, 2000, 2500])
13     densidades_isa = np.array([1.225, 1.167, 1.112, 1.058, 1.007, 0.957])
14
15     # 4. Telemetria del Radar: Tiempo (s), Posicion X (m) y Posicion Y (m)
16     t_tel = np.linspace(0, 8, 17)
17     x_tel = np.array
18     ([0,48,93,137,178,218,256,292,326,359,390,419,448,474,500,524,547])
19     y_tel = np.array
20     ([0,85,160,225,280,325,360,385,400,405,400,385,360,325,280,225,160])
```

Desarrollo de la Práctica

Apartado 1: Calibración de la plataforma de lanzamiento (Sistema Lineal)

La sonda cuenta con un acelerómetro triaxial cuyas lecturas en voltaje $\vec{V}$ están acopladas debido a tolerancias de fabricación. La relación entre las aceleraciones reales $\vec{a} = (a_x, a_y, a_z)^T$ y el voltaje medido viene dada por el sistema lineal: $M \cdot \vec{a} = \vec{V}$.

- **Tarea:** Implementad la resolución numérica de este sistema (usando `scipy.linalg` o `numpy.linalg`) para hallar el vector de aceleración inicial $\vec{a}$ en m/s^2 .

Apartado 2: Caracterización Aerodinámica (Regresión Lineal)

En régimen subsónico, la fuerza de resistencia aerodinámica F_D es proporcional a la presión dinámica $q = \frac{1}{2}\rho v^2$. La constante de proporcionalidad es el producto del coeficiente de resistencia por el área de referencia: $F_D = (C_D A) \cdot q$.

- **Tarea:** Realizad un ajuste por mínimos cuadrados (regresión lineal) a los datos del túnel de viento para calcular el parámetro geométrico-aerodinámico $(C_D A)$.

Apartado 3: Modelo Atmosférico Local (Interpolación 1D)

Para los cálculos aerodinámicos en vuelo, necesitamos la densidad del aire a la altitud máxima (apogeo) registrada por el radar ($y \approx 405 \text{ m}$), pero este valor no está directamente en las tablas ISA.

- **Tarea:** Utilizad interpolación por splines cúbicos (mediante `scipy.interpolate`) sobre los datos de la Atmósfera Estándar para estimar la densidad del aire ρ exactamente a $h = 405 \text{ m}$.

Apartado 4: Cinemática a partir de Telemetría (Derivación Numérica)

El radar nos proporciona vectores discretos de posición (x, y) frente al tiempo t . Necesitamos conocer los perfiles de velocidad para evaluar las fuerzas dinámicas.

- **Tarea:** Empleando derivación numérica (diferencias finitas centrales, y progresivas/regresivas en los extremos), calculad la componente horizontal $v_x(t)$ y vertical $v_y(t)$ de la velocidad. Calculad el módulo de la velocidad $v(t) = \sqrt{v_x^2 + v_y^2}$.

Apartado 5: Energía Disipada por la Atmósfera (Integración Numérica)

El trabajo total realizado por la fuerza de resistencia (energía disipada) desde el lanzamiento hasta el último punto de telemetría se define como la integral de la potencia aerodinámica a lo largo del tiempo:

$$W_{drag} = \int_0^{t_f} F_D(t) \cdot v(t) dt = \int_0^{t_f} \left((C_D A) \cdot \frac{1}{2} \rho_{media} v(t)^2 \right) v(t) dt$$

- **Tarea:** Usando la densidad interpolada en el Apartado 3 (como aproximación de ρ_{media}), el parámetro $(C_D A)$ del Apartado 2, y el vector de velocidades $v(t)$ del Apartado 4, calculad esta integral numérica empleando la regla de Simpson o el método de los trapecios (`scipy.integrate`).

Apartado 6: Predicción del Impacto (Búsqueda de Raíces)

El equipo de aerodinámica ha desarrollado un modelo analítico preliminar para la altitud de la sonda durante su fase final de caída (válido para $t > 5 \text{ s}$), dado por la ecuación no lineal:

$$y(t) = 400 - 35(t - 5) - 150e^{-0,2(t-5)}$$

- **Tarea:** Utilizad un método numérico de búsqueda de raíces (Newton-Raphson, Bisección o `scipy.optimize.root_scalar`) para encontrar el instante exacto de tiempo $t_{impacto}$ en el que la sonda tocará el suelo ($y(t) = 0$). Justificad vuestra elección del intervalo o punto inicial (seed).

Requisitos del Dashboard (GUI)

En lugar de generar múltiples ventanas con gráficas estáticas aisladas, el alumno deberá integrar la visualización de todos los apartados en una única Interfaz Gráfica de Usuario (GUI) interactiva programada **exclusivamente con matplotlib** (haciendo uso del módulo `matplotlib.widgets`).

- **Navegación:** Cread un menú (usando `RadioButtons` o varios `Button`) que permita al usuario cambiar la vista principal para alternar entre los resultados de los diferentes apartados.
- **Interactividad Dinámica:** Al menos en uno de los análisis (ej. la regresión o la predicción de impacto), incluid un `Slider` que permita modificar un parámetro de entrada y observar cómo se actualiza la gráfica en tiempo real.
- **Resultados Numéricos:** Utilizad comandos de texto de matplotlib (`ax.text`) para mostrar en pantalla las soluciones numéricas (ej. la matriz de aceleración calculada o la energía disipada) junto a las gráficas correspondientes.

Requisitos de Entrega

El código debe entregarse en diferentes archivos `*.py` que se llaman desde un archivo `main.py`. Debe estar correctamente comentado y modularizado (utilizad funciones para estructurar el código). **Es obligatorio incluir sentencias `print()` (o mostrar el texto en la propia GUI) que acompañen a cada resultado numérico con una breve frase interpretando su sentido físico.**

2. Análisis Numérico y Telemetría del Rendimiento de un Turbo-propulsor

Contexto Físico y Objetivo

El departamento de propulsión de su compañía aeroespacial ha finalizado la primera campaña de ensayos en banco de pruebas de un nuevo motor turbopropulsor diseñado para aviación regional. La próxima semana, el equipo de ingeniería deberá realizar una **presentación oral ejecutiva** ante la junta directiva para defender la viabilidad técnica y el rendimiento aerotermodinámico del motor.

Para dar soporte visual y técnico a esta presentación, se le ha encomendado el desarrollo de un *dashboard* interactivo. Este panel de control no solo debe procesar los datos de telemetría sin procesar utilizando métodos numéricos rigurosos, sino que debe permitir al ingeniero ponente interactuar con las gráficas en tiempo real durante la presentación para responder a posibles preguntas de la junta. El objetivo de esta práctica es implementar los seis análisis numéricos requeridos utilizando operaciones vectorizadas y empaquetar los resultados en una Interfaz Gráfica de Usuario (GUI) interactiva.

Datos de Partida

A continuación, se proporcionan los datos experimentales simulados que deberá cargar en su script como *arrays* de *numpy*.

```
1      from numpy import array, exp, pi, sin, linspace
2
3      # 1. Calibracion: Matriz de interferencia de la balanza (C) y voltajes leídos
4      # (V)
5      C_matrix = array([[10.5, -0.2], [-0.1, 8.4]]) # [mV/kN, mV/kNm]
6      V_vector = array([250.3, 198.5])             # [mV]
7
8      # 2. Regresion: Potencia en el eje (kW) y Flujo de combustible (kg/h)
9      P_ensayo = array([500.0, 750.0, 1000.0, 1250.0, 1500.0])
10     m_dot_f = array([145.2, 208.5, 275.1, 335.8, 398.4])
11
12     # 3. Interpolacion: Atmosfera Estandar (Altitud en m, Temperatura en K)
13     h_tabla = array([0, 2000, 4000, 6000, 8000, 10000])
14     T_tabla = array([288.15, 275.15, 262.15, 249.15, 236.15, 223.15])
15
16     # 4. Parametros para la Ecuacion No Lineal
17     M_critico = 0.82
18     D_helice = 2.5 # Diametro en metros
19     RPM_crucero = 1200 # Revoluciones por minuto
20     V_vuelo = 150 # Velocidad de avance en m/s
21     gamma = 1.4 # Coeficiente adiabatico del aire
22     R_g = 287.05 # Constante de los gases ideales [J/(kg K)]
23
24     # 5 y 6. Telemetria de arranque: Tiempo (s), RPM y Par motor (Nm)
25     t telemetria = linspace(0, 10, 200)
26     RPM telemetria = 1500*(1-exp(-0.5 * t telemetria))+ 15*sin(2*pi*t telemetria)
27     Par_motor = 2000 * (1-exp(-0.8 * t telemetria))
```

Desarrollo Matemático

El script deberá resolver de forma secuencial y estructurada los siguientes 6 apartados, utilizando exclusivamente funciones y vectorización de *numpy*.

1. Resolución de un sistema lineal (Calibración)

La balanza aerodinámica acopla las lecturas de Empuje (T) y Par (Q). El modelo linealizado de la celda de carga es $\mathbf{C} \cdot \vec{F} = \vec{V}$, donde $\vec{F} = [T, Q]^T$.

- **Tarea:** Resuelva el sistema de ecuaciones para obtener el vector de fuerzas $\vec{F}$ (Empuje en kN y Par en kNm).

2. Regresión Lineal (Consumo Específico)

El consumo de combustible ($\dot{m}_f$) presenta un comportamiento aproximadamente lineal respecto a la potencia (P) en régimen de crucero.

- **Tarea:** Ajuste el modelo $\dot{m}_f = a \cdot P + b$ por mínimos cuadrados. Calcule los coeficientes y determine el valor estimado de flujo de combustible para una potencia de diseño de 1100 kW.

3. Interpolación 1D (Propiedades Atmosféricas)

Para evaluar el rendimiento en altitudes no ensayadas, es necesario estimar la temperatura ambiente.

- **Tarea:** Utilice interpolación lineal para definir una función que calcule la temperatura $T(h)$ a una altitud de 5250 m basándose en los datos tabulados (`h_tabla`, `T_tabla`).

4. Ecuación No Lineal (Mach Crítico en Punta de Pala)

El rendimiento de la hélice cae drásticamente si el número de Mach en la punta de la pala alcanza un valor crítico. El número de Mach en la punta de la hélice a una altitud h es:

$$M_{tip}(h) = \frac{\sqrt{V_{vuelo}^2 + (\omega \cdot R)^2}}{\sqrt{\gamma \cdot R_g \cdot T(h)}}$$

Donde ω es la velocidad angular en rad/s y R el radio de la hélice.

- **Tarea:** Encuentre la altitud h (entre 0 y 10000 m) que satisface la ecuación no lineal $M_{tip}(h) - M_{crit} = 0$. *Pista:* Al estar restringido el uso de bucles, evalúe la función en un vector denso de altitudes y encuentre la intersección con el cero utilizando diferencias de signo y aproximación lineal.

5. Derivación Numérica 1D (Aceleración del Eje)

La telemetría de arranque proporciona las RPM a lo largo del tiempo.

- **Tarea:** Convierta las RPM a velocidad angular $\omega(t)$ (rad/s) y utilice esquemas de diferencias finitas centrales para obtener la aceleración angular $\alpha(t) = \frac{d\omega}{dt}$ (rad/s²).

6. Integración Numérica 1D (Trabajo Mecánico)

La potencia desarrollada por el motor es el producto del par motor por la velocidad angular: $P(t) = Q(t) \cdot \omega(t)$.

- **Tarea:** Calcule el trabajo mecánico total (energía en Julios) entregado por el motor durante los 10 segundos de la telemetría, integrando numéricamente la potencia $W = \int_0^{t_{final}} P(t) dt$.

Requisitos del Dashboard Interactivo (GUI)

Para la presentación oral, se exige abandonar las figuras estáticas aisladas. Usted deberá programar una única ventana gráfica utilizando `matplotlib` (y su módulo `matplotlib.widgets`) con los siguientes elementos:

1. **Área Principal de Gráficos:** Un sistema de ejes principal (Axes) donde se visualizará la gráfica correspondiente al análisis seleccionado.
2. **Selector de Análisis (RadioButtons):** Un widget de tipo `RadioButtons` situado en el margen izquierdo o inferior, que permita al ponente cambiar instantáneamente la vista entre los 6 análisis planteados (ej. "1. Calibración", "2. Regresión", etc.).
3. **Display de Resultados Numéricos:** Un área reservada (puede ser utilizando `matplotlib.pyplot.text` o un `TextBox` inactivo) donde se muestre claramente la solución numérica del apartado actualmente seleccionado, incluyendo sus unidades físicas correctas.

Criterios de Entrega y Restricciones

- **Vectorización Absoluta:** Se prohíbe taxativamente el uso de bucles `for` o `while` para iterar sobre los arrays de datos. Toda operación debe aprovechar la vectorización de `numpy`.
- **Estructura:** Cada apartado numérico debe estar encapsulado en una función (ej. `def resolver_mach_criti`).
- **Interpretación:** En el código, agregue comentarios (docstrings) breves que expliquen si el orden de magnitud del resultado obtenido tiene sentido físico en el contexto aeronáutico.

3. Telemetría y Dinámica de Vuelo del Cohete Sonda “AeroPy-I”

Contexto Físico y Objetivo

El equipo de propulsión de la agencia aeroespacial ha finalizado con éxito el primer vuelo suborbital de prueba del cohete sonda **AeroPy-I**. Durante el lanzamiento, la computadora de a bordo y los sistemas de telemetría terrestres registraron una gran cantidad de datos crudos. Sin embargo, para validar los modelos teóricos de diseño, es necesario procesar esta información, calibrar los sensores y extraer los parámetros críticos del vuelo (rendimiento del motor, aerodinámica y perfil térmico).

El objetivo de esta práctica es desarrollar un **dashboard interactivo** de análisis de telemetría. Como ingenieros de vuelo, deberéis procesar los datos experimentales en bruto implementando algoritmos de métodos numéricos. **Toda la resolución matemática debe estar estrictamente vectorizada** utilizando la biblioteca **numpy**. El uso de bucles iterativos nativos de Python (**for**, **while**) resultará en una penalización severa, ya que la eficiencia computacional es un requisito indispensable en el procesamiento de telemetría en tiempo real.

Datos de Partida

Copiad las siguientes matrices de datos simulados en vuestro script. Representan las lecturas del banco de pruebas y la telemetría del vuelo:

```
1      import numpy as np
2
3      # 1. Calibracion del sensor principal de empuje (Voltaje [V] vs Fuerza [kN])
4      voltaje_sensor = np.array([0.0, 1.25, 2.45, 3.80, 5.10])
5      fuerza_real = np.array([0.0, 15.2, 31.0, 48.5, 65.3])
6
7      # 2. Reacciones en la plataforma de lanzamiento (Coordenadas apoyos y pesos)
8      # Matriz de geometria estatica (distancias en m) para el balance de masas
9      matriz_estatica = np.array([
10     [1.0, 1.0, 1.0],          # Suma de fuerzas Z
11     [0.0, 1.5, -1.5],         # Momentos en X
12     [-2.0, 1.0, 1.0]          # Momentos en Y
13 ])
14     vector_cargas = np.array([25000.0, 1500.0, -800.0]) # [N, Nm, Nm]
15
16     # 3. Datos de Atmosfera Estandar (Altitud [m], Temperatura [K], Presion [Pa])
17     h_atm = np.array([0, 2000, 4000, 6000, 8000, 10000, 12000, 14000])
18     T_atm = np.array([288.15, 275.15, 262.15, 249.15, 236.15, 223.15, 216.65,
19     216.65])
20     P_atm = np.array([101325, 79501, 61660, 47217, 35651, 26499, 19399, 14170])
21
22     # 4. Telemetria de vuelo: Tiempo [s] y Velocidad [m/s]
23     t_vuelo = np.linspace(0, 60, 1200) # Muestreo a 20 Hz
24     # (Perfil simulado de velocidad de un cohete)
25     v_vuelo = 800 * np.sin(np.pi * t_vuelo / 60) * np.exp(-0.02 * t_vuelo)
```

Desarrollo Matemático

El dashboard deberá contener y visualizar la resolución secuencial de los siguientes 6 problemas:

Apartado 1: Balance de Masas (Sistema Lineal)

Antes del despegue, el cohete descansa sobre tres apoyos en la plataforma. Conociendo la matriz geométrica del sistema y el vector de cargas (peso total y momentos flectores por viento), determinad la fuerza de reacción en cada uno de los tres apoyos resolviendo el sistema lineal $\mathbf{A}\vec{x} = \vec{b}$.

- **Requisito NumPy:** Utilizar un método de la sub-biblioteca de álgebra lineal para encontrar $\vec{x}$.

Apartado 2: Calibración del Motor (Regresión Lineal)

El sensor piezoeléctrico de la tobera emite un voltaje que debe traducirse a Empuje (Fuerza). Encontrad la recta de mejor ajuste $F(V) = m \cdot V + c$ minimizando el error cuadrático medio respecto a los datos de calibración proporcionados.

- **Requisito NumPy:** Calcular el ajuste polinómico de grado 1 de forma vectorizada.

Apartado 3: Perfil Atmosférico (Interpolación 1D)

Asumiendo que la altitud del cohete en función del tiempo viene dada por $h(t) = 300 \cdot t$ (una aproximación lineal para los primeros instantes), estimad la Temperatura ambiente $T(t)$ experimentada por el cohete a lo largo de todo su vector temporal `t_vuelo`.

- **Requisito NumPy:** Interpolan la temperatura usando las tablas atmosféricas dadas.

Apartado 4: Búsqueda del Régimen Transónico (Ecuación No Lineal)

Queremos conocer el instante exacto t_c en el que el cohete alcanza un Número de Mach crítico de $M = 1,2$. Sabiendo que $M(t) = \frac{v_{vuelo}(t)}{a(t)}$ y que la velocidad del sonido es $a(t) = \sqrt{\gamma R T(t)}$ (donde $\gamma = 1,4$ y $R = 287 J/(kg \cdot K)$):

- **Requisito NumPy:** Localizar el cruce por cero de la función $f(t) = M(t) - 1,2$. Podéis usar operaciones condicionales sobre arrays para detectar cambios de signo e interpolar localmente el punto exacto, sin bucles.

Apartado 5: Acelerómetro Virtual (Derivación Numérica)

Calculad la aceleración longitudinal del cohete $a_{long}(t) = \frac{dv}{dt}$ a partir del vector de telemetría de velocidades.

- **Requisito NumPy:** Utilizar diferencias finitas centradas de segundo orden (investigad las funciones de gradiente integradas en numpy).

Apartado 6: Altitud Apogeo (Integración Numérica)

A partir de la curva de velocidad experimental `v_vuelo`, calculad la altitud total alcanzada a los $t = 60$ s mediante la integración del área bajo la curva $h_{total} = \int_0^{60} v(t) dt$.

- **Requisito NumPy:** Aplicar la regla del trapecio de forma puramente matemática con arrays o usar la función específica de integración acumulada de numpy.

Requisitos de la Interfaz Gráfica (GUI)

Queda **estrictamente prohibido** generar múltiples ventanas estáticas. Se debe entregar una (y solo una) ventana interactiva haciendo uso del módulo `matplotlib.widgets`.

1. **Navegación:** La figura debe reservar un panel a la izquierda que contenga un `RadioButtons`. Este botón radial permitirá al usuario seleccionar qué apartado (del 1 al 6) se está visualizando en el eje principal.
2. **Interactividad:** Añadid un `Slider` que controle un parámetro físico (por ejemplo, permitir al usuario variar el Mach crítico buscado entre 0.8 y 2.5 en el Apartado 4, actualizándose la gráfica automáticamente).
3. **Reporte Numérico:** El resultado numérico escalar de cada apartado (ej. la altitud máxima, o el instante t_c) debe imprimirse limpiamente superpuesto en la figura mediante un `TextBox` inactivo o un texto estático (`ax.text`) que se actualice dinámicamente.

Criterios de Entrega

- **Vectorización:** Un código que contenga la palabra clave `for` iterando sobre arrays de tiempo o datos resultará en un Insuficiente. automático en la categoría de programación. Pensad en arrays completos.
- **Calidad:** El código debe seguir PEP 8, estar modularizado (una función de Python por apartado) y correctamente comentado.
- **Interpretación:** Cada función debe imprimir en la terminal (además de en la GUI) una frase que interprete el resultado. Por ejemplo: *"La matriz de calibración indica un sesgo de X Newtons, lo que es físicamente consistente con el peso en vacío del sensor."*

4. Senda de Aterrizaje y Dinámica de Frenado de una Aeronave Comercial

Contexto Físico y Objetivo

La fase de aproximación, aterrizaje (touchdown) y frenado (roll-out) constituye uno de los segmentos más críticos en la operación de cualquier aeronave comercial. Durante esta maniobra, el avión debe capturar la senda de planeo (glide slope) del sistema ILS, atravesar la capa límite atmosférica donde el perfil de vientos varía drásticamente, y finalmente disipar su energía cinética en la pista mediante sistemas de frenado aerodinámicos y mecánicos.

El objetivo de esta práctica es desarrollar una herramienta computacional de análisis de telemetría que permita a la oficina técnica evaluar el desempeño de un aterrizaje. Como ingenieros, no solo debéis calcular los parámetros físicos clave de la maniobra, sino también integrar vuestros resultados en un **Panel de Control (Dashboard) Interactivo**. Para ello, se exigirá un uso riguroso de operaciones vectorizadas con `numpy` para garantizar el máximo rendimiento computacional, evitando los bucles tradicionales de Python (`for`, `while`) siempre que sea matemáticamente posible.

Datos de Partida (Telemetría Simulada)

Copiad y pegad el siguiente bloque de datos en vuestro script. Estos arrays contienen las mediciones de los sensores de a bordo y de las estaciones terrestres.

```
1      import numpy as np
2
3      # Datos para el Sistema Lineal (Geometria del tren y pesos)
4      # x_cg, peso_total, momentos en x e y
5      params_masa = np.array([18.5, 75000 * 9.81, 13600000.0, 0.0])
6      matriz_geometria = np.array([
7          [1.0, 1.0, 1.0],          # Suma de fuerzas verticales
8          [3.2, 20.1, 20.1],        # Brazos de palanca eje X
9          [0.0, -4.5, 4.5]          # Brazos de palanca eje Y
10     ])
11
12     # Datos para Regresion Lineal (Telemetria ILS: Distancia a pista vs Altitud)
13     x_ils = np.linspace(-10000, -500, 200)
14     # Altitud con ruido de medicion del radar
15     h_ils = 0.0524 * np.abs(x_ils) + 15.0 + np.random.normal(0, 8, 200)
16
17     # Datos para Interpolacion 1D (Perfil de viento en la capa limite)
18     altitudes_viento = np.array([0, 50, 100, 200, 500, 1000]) # [m]
19     velocidad_viento = np.array([2.1, 4.5, 7.2, 8.5, 12.0, 15.3]) # [m/s]
20
21     # Datos para Derivacion e Integracion (Dinamica de frenado)
22     tiempo_frenado = np.linspace(0, 35, 1000) # [s]
23     # Perfil de velocidad [m/s]
24     velocidad_frenado = 70.0 * np.exp(-0.04 * tiempo_frenado) - 1.2 * tiempo_frenado
25     velocidad_frenado[velocidad_frenado < 0] = 0
```

Desarrollo Matemático

Deberéis resolver secuencialmente los siguientes 6 problemas matemáticos implementando los algoritmos numéricos correspondientes.

1. Sistema Lineal de Ecuaciones: Reparto de Cargas

Antes del aterrizaje, es vital conocer la distribución del peso en el tren de morro (R_m) y los trenes principales (R_{pi} , R_{pd}). Plantead y resolved el sistema lineal $\mathbf{Ax} = \mathbf{b}$, donde $\mathbf{x} = [R_m, R_{pi}, R_{pd}]^T$,

utilizando la matriz de geometría y el vector de cargas estáticas `params_masa` (Suma de fuerzas = Peso, Suma de momentos en X e Y).

Requisito: Utilizar `np.linalg.solve`.

2. Regresión Lineal: Calibración de la Senda de Planeo

Las normativas de la OACI exigen que el ángulo de descenso (γ) sea de aproximadamente 3° . Utilizad los datos de telemetría del ILS (`x_ils`, `h_ils`) para realizar un ajuste por mínimos cuadrados a una recta $h = mx + n$.

Requisito: Implementar las ecuaciones normales de la regresión lineal utilizando operaciones matriciales de `numpy` (ej. productos punto e inversas). Calculad el ángulo real de descenso $\gamma = \arctan(m)$ en grados.

3. Interpolación 1D: Viento en la Altura de Decisión

La altitud de decisión (Decision Height) para este aterrizaje es de 60 metros. Determinad la velocidad del viento cruzado a esa altitud exacta utilizando una interpolación lineal sobre los arrays `altitudes_viento` y `velocidad_viento`.

Requisito: Utilizar la función vectorizada `np.interp`.

4. Ecuación No Lineal: Instante de Velocidad de Rodaje

Una vez que el avión toca pista, debe alcanzar una velocidad segura de rodaje ($V_{taxi} = 15$ m/s) para abandonar la pista. El perfil de velocidad durante el frenado responde a una ecuación trascendental simplificada:

$$V(t) = 70e^{-0,04t} - 1,2t \quad (1)$$

Requisito: Implementar el **Método de Newton-Raphson** para encontrar la raíz de la función $f(t) = V(t) - 15 = 0$. Podéis usar un bucle `while` exclusivamente para las iteraciones de este método, tomando una semilla inicial $t_0 = 10$.

5. Derivación Numérica 1D: Perfil de Desaceleración

La comodidad de los pasajeros exige que la desaceleración no supere $0,3g$ ($\approx 2,94$ m/s²). Utilizando los arrays `tiempo_frenado` y `velocidad_frenado`, calculad la aceleración instantánea $a(t) = \frac{dV}{dt}$.

Requisito: Aplicar el método de **Diferencias Finitas Centrales** vectorizando las operaciones con el *slicing* de arrays en `numpy` (ej. `array[2:] - array[:-2]`).

6. Integración Numérica 1D: Distancia de Pista Consumida

Para garantizar que el avión no se sale de la pista, calculad la distancia total recorrida desde el instante del touchdown hasta que la aeronave se detiene por completo ($V = 0$). Sabiendo que $S = \int V(t)dt$.

Requisito: Programar la **Regla del Trapecio** de forma puramente vectorizada (sin usar bucles ni `np.trapz`), operando directamente sobre los arrays.

Requisitos del Dashboard (Interfaz Gráfica)

No se aceptarán scripts que generen 6 ventanas de gráficas estáticas consecutivas. Debéis construir una única Interfaz Gráfica de Usuario (GUI) interactiva utilizando `matplotlib` y su módulo `matplotlib.widgets`.

1. **Diseño de la Ventana:** La figura (`plt.figure`) debe tener un área principal para mostrar la gráfica y un panel lateral o inferior para los controles.

2. **Interactividad (RadioButtons):** Implementad un widget de tipo `RadioButtons` que permita al usuario alternar entre tres vistas lógicas:
- *Aproximación:* Muestra la gráfica de regresión del ILS y marca el ángulo calculado.
 - *Atmósfera:* Muestra el perfil de vientos y un punto rojo interpolado en la Altitud de Decisión.
 - *Frenado:* Muestra dos sub-gráficas (velocidad y aceleración), sombreando el área de la distancia integrada y marcando t_{taxi} de Newton-Raphson.
3. **Salida Numérica:** Utilizad anotaciones (`plt.text`) dinámicas en la gráfica o en un eje de texto reservado para mostrar en pantalla las soluciones numéricas requeridas (Cargas, Ángulo, Viento, t_{taxi} , Distancia) que se actualicen según la vista seleccionada.

Criterios de Entrega

- **Vectorización:** El uso de bucles iterativos (`for`, `while`) penalizará fuertemente la nota, con la única excepción de la iteración del método de Newton-Raphson.
- **Interpretación:** El código debe incluir la función `print()` proporcionando una evaluación física de los resultados obtenidos (¿Es seguro el aterrizaje? ¿Supera la desaceleración máxima?).
- **Legibilidad:** Respetar la normativa PEP 8.

5. Análisis Orbital y de Telecomunicaciones de la Constelación Starlink

Contexto Físico y Objetivo

La constelación **Starlink** de SpaceX representa el mayor despliegue de satélites de órbita terrestre baja (LEO) de la historia, diseñados para proporcionar conectividad global de banda ancha y baja latencia. Tras el despliegue inicial desde el vehículo de lanzamiento Falcon 9, los satélites utilizan propulsores de efecto Hall (iónicos) de argón/criptón para elevar su órbita (orbit-raising) hasta su altitud operativa final (típicamente entre 540 km y 550 km). Durante esta fase crítica, la telemetría continua y el análisis del entorno atmosférico son fundamentales para garantizar la supervivencia del satélite y la calidad del enlace de comunicaciones (bandas Ku y Ka).

El objetivo de esta práctica es desarrollar un **dashboard interactivo** que procese y analice los datos de telemetría simulados de un satélite Starlink durante una maniobra de ascenso de 120 minutos. Se requiere que implementéis herramientas de análisis numérico para evaluar la posición, velocidad, entorno aerodinámico y capacidad de transferencia de datos del satélite. Todo el análisis debe realizarse bajo el paradigma de **programación vectorizada**, demostrando un dominio absoluto de la biblioteca computacional moderna.

Datos de Partida

Copiad y pegad el siguiente bloque de código en vuestro script. Contiene los datos simulados de telemetría (vectores de tamaño $N = 1000$) y las matrices de calibración.

```
1      import numpy as np
2
3      # Semilla para reproducibilidad
4      np.random.seed(42)
5
6      # Tiempo de la mision [minutos]
7      t = np.linspace(0, 120, 1000)
8
9      # Altitud registrada por telemetria [km] (con ligeras oscilaciones)
10     h = 250 + 1.2 * t + 5 * np.sin(0.1 * t)
11
12     # Potencia de senal recibida en la estacion terrestre (Gateway) [dBm]
13     P_rx = -40 - 0.15 * t + 1.5 * np.random.randn(1000)
14
15     # Datos atmosfericos tabulados de la U.S. Standard Atmosphere
16     h_tab = np.array([200, 300, 400, 500, 600]) # Altitud [km]
17     rho_tab = np.array([2.789e-10, 1.95e-11, 2.82e-12, 5.21e-13, 1.13e-13])
18
19     # Matriz de acoplamiento de la antena phased-array y retardos medidos
20     A_cal = np.array([[1.20, 0.30, 0.10],
21                       [0.20, 1.05, 0.20],
22                       [0.10, 0.15, 1.10]])
23     b_cal = np.array([15.2, 12.5, 14.8])
```

Desarrollo Matemático

Implementad los siguientes 6 apartados secuenciales utilizando **exclusivamente funciones de NumPy**. El uso de bucles `for` o `while` nativos resultará en una fuerte penalización.

1. Resolución de un sistema lineal (Calibración de la Antena)

Las estaciones terrestres de Starlink utilizan antenas de arreglo en fase (phased-array). Durante la recepción, existe un acoplamiento electromagnético entre tres sectores principales del arreglo. El retardo real de propagación de la señal $\mathbf{x} = [x_1, x_2, x_3]^T$ se relaciona con el retardo aparente medido $\mathbf{b}_{\text{cal}}$ mediante la matriz de interferencia $\mathbf{A}_{\text{cal}}$ según la ecuación: $\mathbf{A}_{\text{cal}}\mathbf{x} = \mathbf{b}_{\text{cal}}$.

- **Se pide:** Resolver el sistema lineal algebraico para hallar el vector de retardos verdaderos $\mathbf{x}$ empleando las rutinas de álgebra lineal de NumPy.

2. Regresión lineal (Modelo de Atenuación de Señal)

La potencia de la señal de telemetría P_{rx} sufre una atenuación a lo largo de la maniobra debido al aumento de la distancia y las condiciones ionosféricas.

- **Se pide:** Ajustar los datos dispersos (t, P_{rx}) a un modelo lineal $P_{rx}(t) = m \cdot t + c$. Calculad la pendiente m (tasa de degradación en dBm/min) y la ordenada en el origen c mediante un ajuste de mínimos cuadrados. Mostrad la recta de regresión superpuesta a los datos ruidosos.

3. Interpolación 1D (Perfil de Densidad Atmosférica)

Para predecir el frenado aerodinámico (drag) en régimen de flujo molecular libre, es necesario conocer la densidad del aire en cualquier instante de la trayectoria.

- **Se pide:** Interpolarse el vector de altitud continua h utilizando los datos tabulados discretos `h.tab` y `rho.tab` para obtener un vector continuo de densidad $\rho(t)$ correspondiente a cada instante de la misión. *Sugerencia: Debido al comportamiento exponencial de la atmósfera, se valorará positivamente interpolarse en escala logarítmica.*

4. Resolución de una Ecuación No Lineal (Altitud Crítica)

La resistencia atmosférica decae exponencialmente. SpaceX define un umbral de seguridad donde el “drag” es considerado insignificante, permitiendo el despliegue total de los paneles solares. Asumiremos que este umbral se alcanza a una altitud exacta de $h_{crit} = 375$ km.

- **Se pide:** Encontrar el instante exacto de tiempo t_{crit} en el que el satélite alcanza dicha altitud, resolviendo la raíz de la ecuación no lineal $h(t) - 375 = 0$. Emplead un método vectorizado (ej. localizar el cambio de signo temporal mediante diferencias y aplicar interpolación local lineal).

5. Derivación Numérica 1D (Cinemática del Satélite)

A partir de los datos de altitud $h(t)$, debemos caracterizar la tasa de ascenso.

- **Se pide:** Calcular el vector de velocidad radial $v(t) = \frac{dh}{dt}$ (en m/s) y la aceleración radial $a(t) = \frac{d^2h}{dt^2}$ (en m/s²) utilizando esquemas numéricos de diferencias finitas centrales. Aseguraos de tratar adecuadamente las condiciones en los bordes del array.

6. Integración Numérica 1D (Telemetría de Datos Transmitidos)

La capacidad de ancho de banda del satélite depende de la relación señal/ruido. Asumiremos un modelo empírico donde la tasa de bits instantánea en Megabytes por minuto se aproxima exponencialmente a partir de la potencia de señal recibida: $R(t) = 10 \cdot 10^{\frac{P_{rx}(t)+60}{20}}$ [MB/min].

- **Se pide:** Calcular el volumen total de datos V (en Gigabytes) transmitidos con éxito a la Tierra durante toda la misión, aplicando integración numérica (ej. Regla del Trapecio compuesto) sobre la curva $R(t)$:

$$V = \int_0^{t_{final}} R(t) dt$$

Requisitos del Dashboard (GUI Interfaz Gráfica)

La práctica debe culminar en una única ventana interactiva programada íntegramente con `matplotlib` y su módulo `matplotlib.widgets`.

1. **Panel de Control:** En la zona lateral o inferior de la figura, debéis incluir un widget `RadioButtons` que permita al usuario cambiar dinámicamente entre las 6 gráficas correspondientes a los 6 cálculos realizados.
2. **Interactividad:** Incluir al menos un widget `Slider`. Por ejemplo, un deslizador que permita modificar el umbral crítico de altitud del apartado 4 (entre 300 y 450 km) y que actualice instantáneamente en la gráfica la posición del punto calculado.
3. **Resultados Numéricos:** Utilizad la función `plt.text` (o equivalente) anclada en un rincón de la figura gráfica para mostrar en texto claro el resultado matemático fundamental del apartado activo (ej. "*Tasa de degradación: -0.15 dBm/min*", o "*Total de datos: 4.2 GB*").

Criterios de Entrega

- Se entregará un único script `.py`.
- **Tolerancia Cero a Bucles:** Todo cálculo en arrays debe hacerse mediante *broadcasting* o funciones nativas de `numpy`. Se restará hasta un 50% de la nota matemática por usar `for` o `while` innecesarios.
- El código debe seguir las normas de estilo PEP-8, con un código modularizado (uso de funciones `def` para separar la matemática de la interfaz gráfica).
- Se valorará profundamente la interpretación física de los resultados impresa tanto en los comentarios del código como en la propia interfaz visual.

6. Análisis Telemétrico y Aerodinámico del Lanzamiento de una Sonda a Marte

Contexto Físico y Objetivo

El departamento de misiones espaciales ha registrado los datos de telemetría del primer vuelo de prueba atmosférico del vehículo de lanzamiento pesado *Ares-I*, cuyo propósito es insertar una sonda científica en una trayectoria de transferencia hacia Marte. Durante la primera etapa del ascenso a través de la atmósfera terrestre, resulta crítico para la integridad estructural y el control de vuelo monitorizar el reparto de masas, las cargas aerodinámicas (como la presión dinámica) y el rendimiento propulsivo.

El objetivo de esta práctica es desarrollar un **Panel de Control de Telemetría (Dashboard) Interactivo** en Python. Deberás procesar los datos brutos del vuelo, aplicando métodos numéricos fundamentales para extraer información física vital de la misión. Como futuros ingenieros aeroespaciales, se exige un código computacionalmente eficiente; por ello, la resolución matemática deberá apoyarse íntegramente en las operaciones vectorizadas de la biblioteca `numpy`, erradicando el uso de bucles nativos ineficientes de Python.

Datos de Partida

Copia el siguiente bloque de código en tu script para inicializar los datos simulados de telemetría y modelos atmosféricos:

```
1      import numpy as np
2
3      # 1. Datos para calibracion del tubo Pitot (Voltaje [V] vs Presion [kPa])
4      V_sensor = np.array([0.5, 1.2, 2.3, 3.1, 4.0, 4.8])
5      P_sensor = np.array([10.1, 25.5, 48.0, 65.2, 83.9, 101.3])
6
7      # 2. Modelo atmosferico terrestre simplificado (Altitud [m] vs Densidad [kg/m
8      ^3])
9      altitudes_tab = np.array([0, 5000, 10000, 15000, 20000, 25000, 30000])
10     densidades_tab = np.array([1.225, 0.736, 0.413, 0.194, 0.088, 0.040, 0.018])
11
12     # 3. Telemetria de vuelo: Tiempo [s] (muestreo a 1 Hz durante 2 min)
13     t_vuelo = np.linspace(0, 120, 121)
14
15     # Perfil de velocidad simulado [m/s] (modelo logistico)
16     v_vuelo = 2500 * (1 / (1 + np.exp(-0.05 * (t_vuelo - 60)))) - 118
```

Desarrollo Matemático

El Dashboard interactivo deberá constar de 6 modos de visualización distintos, correspondientes a los siguientes 6 análisis numéricos secuenciales. Debes organizar tu código en funciones (ej. `def calcular_altitud(t, v):`) antes de conectarlas a la interfaz.

Sistema Lineal de Ecuaciones: Balance de Masas de Propelente

La primera etapa del cohete alberga tres tanques de propergol líquido de masas m_1 , m_2 y m_3 . Los sensores indican que la masa total actual de propergol es de 50000 kg. Por cuestiones de estabilidad aerodinámica, el ordenador de vuelo requiere que la suma de la masa de los dos primeros tanques supere a la del tercero en 15000 kg. Además, los flujos máscicos exigen que la masa contenida en el tanque 1 sea exactamente el doble que la del tanque 2. Plantea este escenario como un sistema $A\vec{x} = \vec{b}$:

$$\begin{cases} m_1 + m_2 + m_3 = 50000 \\ m_1 + m_2 - m_3 = 15000 \\ m_1 - 2m_2 = 0 \end{cases}$$

Tarea: Resuelve el sistema utilizando el módulo de álgebra lineal de `numpy` para encontrar la masa en cada tanque.

Regresión Lineal: Calibración del Sensor de Presión Dinámica

Para medir la carga aerodinámica, la sonda utiliza transductores analógicos. Tenemos pares de datos de calibración en laboratorio que relacionan la señal del sensor en Voltios (V) con la presión en kiloPascuales (P).

Tarea: Implementa un ajuste por mínimos cuadrados (puedes programar la ecuación normal matricial $\vec{\theta} = (X^T X)^{-1} X^T \vec{y}$ o usar funciones de ajuste vectorizadas) para encontrar la recta de calibración del sensor:

$$P(V) = \theta_1 V + \theta_0$$

Integración Numérica 1D: Reconstrucción de la Trayectoria (Altitud)

El radar de tierra ha perdido la señal de la posición altimétrica, pero disponemos del vector de velocidades $v(t)$ (array `v_vuelo`).

Tarea: Obtén el perfil de altitud $h(t)$ integrando la velocidad respecto al tiempo mediante la **regla del trapecio múltiple o acumulada**, utilizando única y exclusivamente el array de datos y operaciones matriciales (no iteraciones paso a paso).

$$h(t_k) = \int_0^{t_k} v(\tau) d\tau$$

Interpolación 1D: Estimación de la Densidad Atmosférica

Con el array $h(t)$ calculado en el apartado anterior, ahora conocemos la altitud en cada segundo del vuelo. Sin embargo, nuestro modelo atmosférico (`altitudes.tab`) solo ofrece datos cada 5000 m.

Tarea: Utiliza un algoritmo de interpolación lineal en 1D (como `np.interp`) para construir un nuevo array con la densidad atmosférica $\rho(t)$ correspondiente a cada instante temporal del vuelo.

Derivación Numérica 1D: Cálculo de Aceleración Estructural

La integridad de la carga de pago requiere que la aceleración axial no supere ciertos límites críticos.

Tarea: A partir del vector discretizado de telemetría `v_vuelo`, calcula el perfil de aceleración $a(t)$ empleando el método de **diferencias finitas centrales** para los puntos interiores (y diferencias adelantadas/atrasadas de primer orden para los extremos del array). Todo de forma estrictamente vectorizada.

$$a_i = \frac{v_{i+1} - v_{i-1}}{t_{i+1} - t_{i-1}}$$

Ecuación No Lineal: Instante Crítico del Vuelo (Mach target)

Es vital conocer el tiempo exacto t_{crit} en el que el cohete alcanza una velocidad diana específica (por defecto, $v_{target} = 340$ m/s, equivalente a Mach 1 para atravesar la barrera del sonido). Para este apartado asume que la velocidad sigue estrictamente la función teórica continua:

$$v(t) = 2500 \left(\frac{1}{1 + e^{-0,05(t-60)}} \right) - 118$$

Tarea: Implementa el método de **Newton-Raphson** o el método de la **Bisección** para encontrar la raíz temporal de la función residual: $f(t) = v(t) - v_{target} = 0$.

Requisitos del Dashboard Interactivo (GUI)

Tu práctica no debe imprimir texto genérico en consola ni generar gráficas estáticas aisladas. Deberás orquestar una ventana interactiva de `matplotlib` (utilizando el módulo `matplotlib.widgets`) que contenga:

1. **RadioButtons:** Un menú lateral de botones radiales que permita al usuario alternar visualmente entre los 6 apartados de la práctica. Al pulsar un botón, la gráfica principal debe actualizarse limpiando los ejes (`ax.clear()`) y plotando la curva pertinente con sus unidades físicas correctas.
2. **Slider Interactivo:** Exclusivamente en la vista del Apartado 6, añade un `Slider` que controle el parámetro de velocidad diana v_{target} (rango de 300 m/s a 1000 m/s). Al mover el slider, el algoritmo de búsqueda de raíces debe recalcular t_{crit} en tiempo real, actualizando dinámicamente un punto indicador en la gráfica y el texto informativo.
3. **Display Numérico:** Dentro del lienzo de cada gráfica, utiliza herramientas como `ax.text` para mostrar el resultado numérico exacto y una breve frase que explique el significado físico de ese resultado para la misión.

Criterios de Entrega y Defensa Oral

- **Eficiencia (Vectorización):** Se evaluará negativamente la presencia de bucles `for` o `while` en operaciones operables vectorialmente por `numpy`. (Excepción: el bucle necesario para el algoritmo iterativo del Apartado 6).
- **Defensa Oral:** Deberás ejecutar la interfaz gráfica ante el profesor, demostrar su dinamismo utilizando el `Slider`, y responder a preguntas técnicas tanto de tu código como de la física subyacente.

7. Análisis de Telemetría y Ascenso de una Sonda Meteorológica Estratosférica

Contexto Físico y Objetivo

El estudio de la atmósfera terrestre es fundamental en la ingeniería aeroespacial, tanto para la predicción meteorológica como para el cálculo de trayectorias de vehículos de lanzamiento. En esta práctica, formáis parte del equipo de análisis de la misión *AeroSonde-I*, un globo meteorológico de gran altitud instrumentado con sensores de presión, temperatura, acelerómetros y un sistema de telemetría.

Durante su reciente lanzamiento, la sonda transmitió un paquete de datos crudos (raw data) que presenta ruido, intervalos irregulares y requiere calibración. Vuestro objetivo es procesar matemáticamente esta telemetría para reconstruir la cinemática del vuelo y las propiedades atmosféricas. Para la **defensa oral** de la práctica, deberéis programar un *Dashboard* interactivo que permita al equipo de control de misión visualizar los seis análisis matemáticos secuenciales, demostrando dominio absoluto de la computación numérica vectorizada.

Datos de Partida

Copien y peguen el siguiente bloque de datos en su script. Queda **terminantemente prohibido** el uso de bucles `for` o `while` de Python para el procesamiento de estos arrays; toda operación debe estar vectorizada con `numpy`.

```
1      import numpy as np
2
3      # 1. Matriz de calibracion del anemometro 3D (Voltajes)
4      # y vector de velocidades de viento de referencia (m/s)
5      voltajes_anemometro = np.array([[1.2, 0.3, 0.1], [0.2, 1.5, 0.4], [0.1, 0.2,
6      1.1]])
7      viento_referencia = np.array([15.5, -5.2, 3.1])
8
9      # 2. Datos de calibracion del altimetro barometrico: Voltaje (V) vs Presion
10     log (ln(Pa))
11     v_sensor = np.array([0.5, 1.2, 2.0, 2.8, 3.5, 4.1, 5.0])
12     ln_presion = np.array([11.5, 10.8, 9.9, 9.1, 8.2, 7.5, 6.4])
13
14     # 3. Telemetria irregular: Tiempos de recepcion (s) y Altitud registrada (m)
15     t telemetria = np.array([0, 12, 25, 43, 58, 71, 89, 105, 120, 145])
16     altitud_cruda = np.array([500, 650, 820, 1050, 1280, 1500, 1790, 2100, 2350,
17     2800])
18
19     # 4. Perfil de temperatura atmosferica en funcion del tiempo T(t) en grados
20     Celsius
21     # Modelo polinomico ajustado: T(t) = 15 - 0.05*t - 0.001*t^2 + 2e-5*t^3
22
23     # 6. Datos de consumo de potencia del calefactor de a bordo (W) en el tiempo
24     # (Se calculara a partir de los tiempos interpolados en el apartado 3)
```

Desarrollo Matemático

Deberán implementar los siguientes 6 módulos numéricos secuenciales:

1. Sistema Lineal: Calibración del Anemómetro

La sonda cuenta con un anemómetro de tres ejes. La relación entre los voltajes medidos en los ejes V_i y las componentes de la velocidad del viento real W_i obedece a un sistema lineal $A \cdot \vec{c} = \vec{W}_{ref}$, donde A es la matriz de `voltajes_anemometro` y $\vec{W}_{ref}$ es el `viento_referencia`.

- **Tarea:** Resuelvan el sistema lineal para hallar el vector de coeficientes de calibración $\vec{c}$.

2. Regresión Lineal: Calibración Barométrica

El sensor de presión emite un voltaje que es inversamente proporcional al logaritmo de la presión atmosférica.

- **Tarea:** Implementen un ajuste por mínimos cuadrados (regresión lineal) para encontrar la recta $\ln(P) = m \cdot V + n$ que mejor ajuste los datos de `v_sensor` y `ln_presion`.

3. Interpolación 1D: Reconstrucción de la Trayectoria

Por pérdidas de señal, la telemetría se recibió en instantes asimétricos (`t_telemetria`). Para el análisis de vuelo, necesitamos la altitud en intervalos regulares.

- **Tarea:** Creen un nuevo vector temporal t_{reg} desde 0 hasta 145 segundos con 200 puntos equiespaciados. Utilicen interpolación lineal o polinómica de `numpy` o `scipy.interpolate` para estimar la altitud $h(t_{reg})$ en estos nuevos instantes.

4. Ecuación No Lineal: Detección de la Tropopausa

La temperatura de la atmósfera desciende con la altitud hasta alcanzar un punto mínimo y estabilizarse. El modelo térmico de la sonda viene dado por:

$$T(t) = 15 - 0,05t - 0,001t^2 + 2 \times 10^{-5}t^3$$

- **Tarea:** Apliquen un método numérico (Bisección o Newton-Raphson) para encontrar el instante de tiempo exacto t_{crit} en el que la temperatura alcanza los -10°C (temperatura de riesgo de congelación para las baterías).

5. Derivación Numérica: Perfil de Velocidad Ascensional

A partir de la altitud interpolada regularmente $h(t_{reg})$ obtenida en el Apartado 3.

- **Tarea:** Calculen la velocidad ascensional $v_z(t) = \frac{dh}{dt}$ utilizando esquemas de diferencias finitas centrales de orden $\mathcal{O}(\Delta t^2)$ (y diferencias hacia adelante/atrás en los extremos).

6. Integración Numérica: Consumo Energético

El calefactor de la sonda tiene un consumo de potencia dependiente de la velocidad y el tiempo estimado por la función: $P(t) = 5 + 0,1 \cdot |v_z(t)| + 0,02 \cdot t$ (en Vatios).

- **Tarea:** Calculen la energía total consumida en Julios $E = \int_0^{145} P(t) dt$ utilizando la regla del Trapecio o Simpson aplicada al vector de potencias calculado sobre t_{reg} .

Requisitos del Dashboard (GUI Interactivo)

Para la **presentación oral**, no se admiten figuras aisladas. Deberán construir un único entorno interactivo utilizando exclusivamente la biblioteca `matplotlib` y su módulo de *widgets* (`matplotlib.widgets`).

1. **Selector de Análisis (RadioButtons):** Un menú lateral que permita al profesor alternar visualmente entre las 6 gráficas correspondientes a los 6 apartados. Al hacer clic en una opción, el eje principal debe limpiarse (`ax.clear()`) y dibujar el análisis correspondiente.
2. **Panel Numérico (Text):** Un área de la figura dedicada a mostrar el texto con las respuestas numéricas del apartado activo (ej. `.Energía total: 1250.3 J`).
3. **Control Interactivo (Slider):** Añadan un deslizador (*Slider*) visible durante el Apartado 4 que permita modificar la "Temperatura Crítica" (entre 0°C y -20°C) y recalcula/redibuje en tiempo real la raíz de la ecuación (el instante t_{crit}).

Criterios de Entrega

- **Vectorización (Obligatoria):** Se auditará el código. El uso injustificado de estructuras de control iterativas (`for`, `while`) para operaciones matemáticas sobre vectores conllevará el suspenso automático de la práctica.
- **Presentación:** La legibilidad del código (PEP-8) y la física detrás de los resultados (impresos en consola o en la GUI) son vitales. Justifiquen si los valores de velocidad o energía calculados tienen sentido en un entorno aeroespacial.